



deeplearning.ai

Convolutional Neural Networks

Computer vision

Computer Vision Problems

Image Classification



64x64

→ Cat? (0/1)

Neural Style Transfer



Object detection

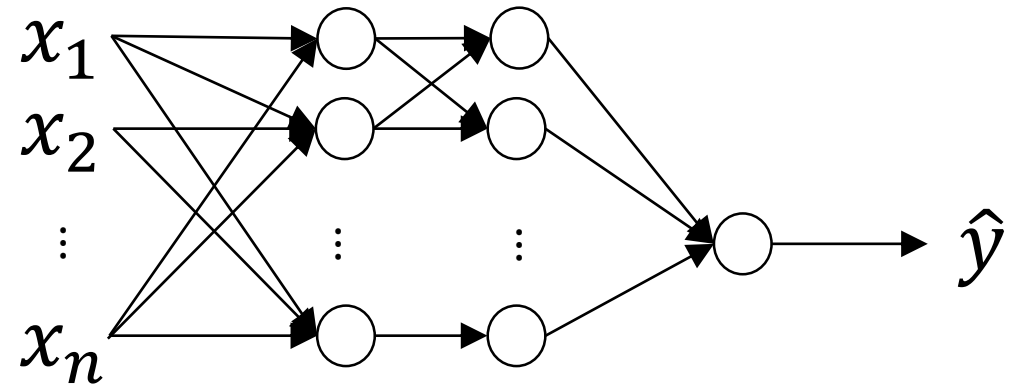


Deep Learning on large images



64x64

→ Cat? (0/1)



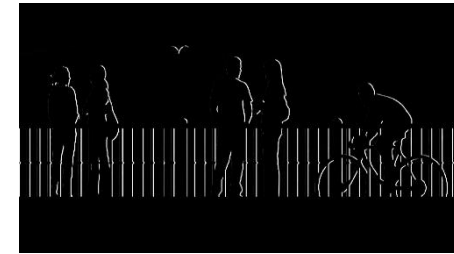


deeplearning.ai

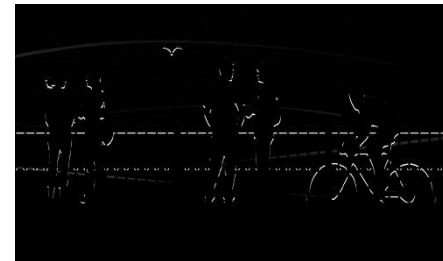
Convolutional Neural Networks

Edge detection example

Computer Vision Problem



vertical edges



horizontal edges

Vertical edge detection

3 ¹	0 ⁰	1 ⁻¹	2 ⁻¹	7 ⁻⁰	4 ⁻¹
1 ¹	5 ⁰	8 ⁻¹	9 ⁻¹	3 ⁻⁰	1 ⁻¹
2 ¹	7 ⁰	2 ⁻¹	5 ⁻¹	1 ⁻⁰	3 ⁻¹
0 ¹	1 ⁰	3 ⁻¹	1 ⁻¹	7 ⁻⁰	8 ⁻¹
4	2	1	6	2	8
2	4	5	2	3	9

*

=

0	-2	-4	-7
-3	-2	-3	-16

Vertical edge detection

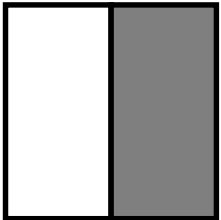
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

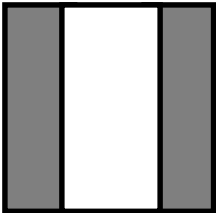
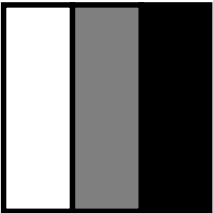
1	0	-1
1	0	-1
1	0	-1

=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



*





deeplearning.ai

Convolutional Neural Networks

More edge
detection

Vertical edge detection examples

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0



*

1	0	-1
1	0	-1
1	0	-1



=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10



*

1	0	-1
1	0	-1
1	0	-1



=

0	-30	-30	0
0	-30	-30	0
0	-30	-30	0
0	-30	-30	0



Vertical and Horizontal Edge Detection

1	0	-1
1	0	-1
1	0	-1

Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

*

1	1	1
0	0	0
-1	-1	-1

=

0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

Learning to detect edges

1	0	-1
1	0	-1
1	0	-1

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

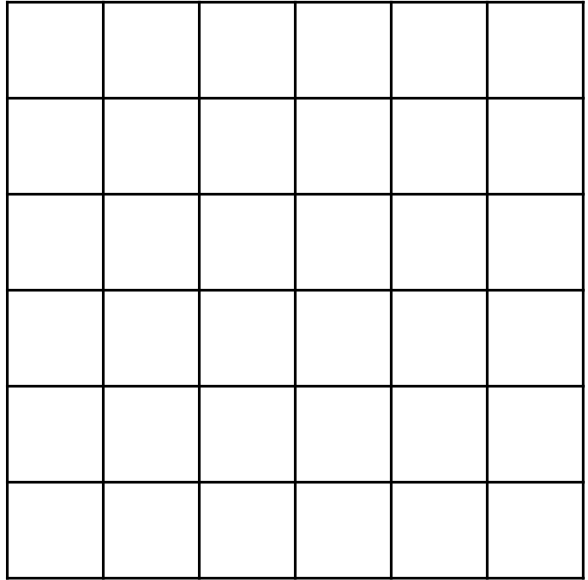


deeplearning.ai

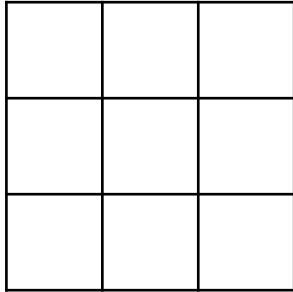
Convolutional Neural Networks

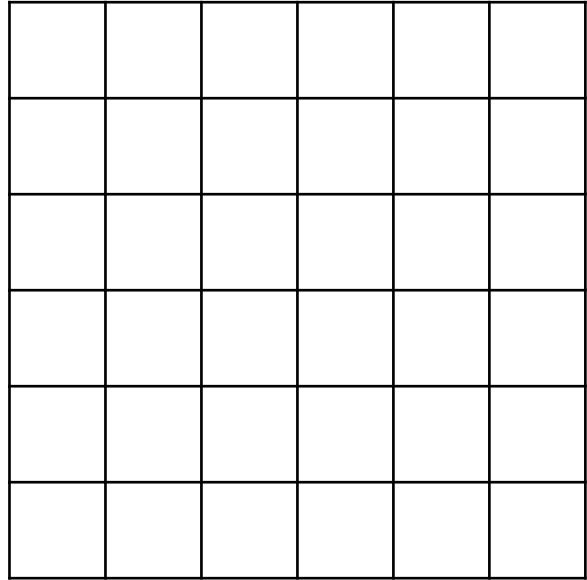
Padding

Padding



*





Valid and Same convolutions

“Valid”:

“Same”: Pad so that output size is the same as the input size.



deeplearning.ai

Convolutional Neural Networks

Strided
convolutions

Strided convolution

2 ³	3 ⁴	7 ³	4 ⁴	6 ³	2 ⁴	9 ⁴
6 ¹	6 ⁰	9 ¹	8 ⁰	7 ¹	4 ⁰	3 ²
3 ⁻³	4 ⁴	8 ³	3 ⁴	8 ³	9 ⁴	7 ⁴
7 ¹	8 ⁰	3 ¹	6 ⁰	6 ¹	3 ⁰	4 ²
4 ⁻³	2 ⁴	1 ³	8 ⁴	3 ³	4 ⁴	6 ⁴
3 ¹	2 ⁰	4 ¹	1 ⁰	9 ¹	8 ⁰	3 ²
0 ⁻¹	1 ⁰	3 ⁻¹	9 ⁰	2 ⁻¹	1 ⁰	4 ³

*

3	4	4
1	0	2
-1	0	3

=

Summary of convolutions

$n \times n$ image $f \times f$ filter

padding p stride s

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

Technical note on cross-correlation vs. convolution

Convolution in math textbook:

2	3	7	4	6	2
6	6	9	8	7	4
3	4	8	3	8	9
7	8	3	6	6	3
4	2	1	8	3	4
3	2	4	1	9	8

*

3	4	5
1	0	2
-1	9	7



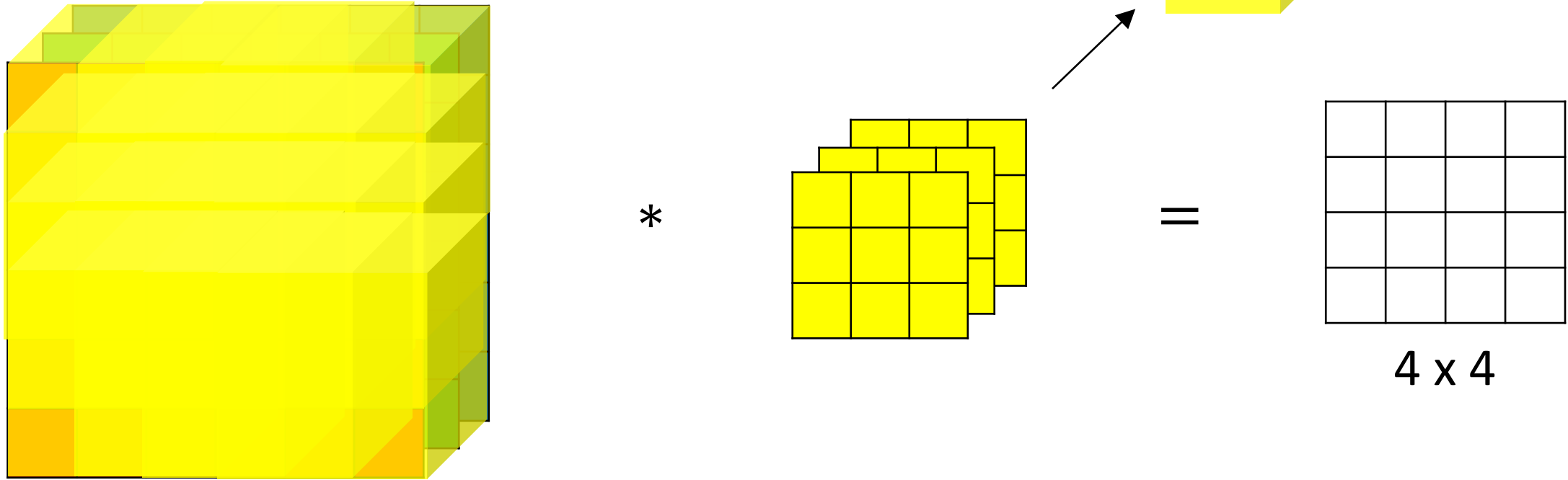
deeplearning.ai

Convolutional Neural Networks

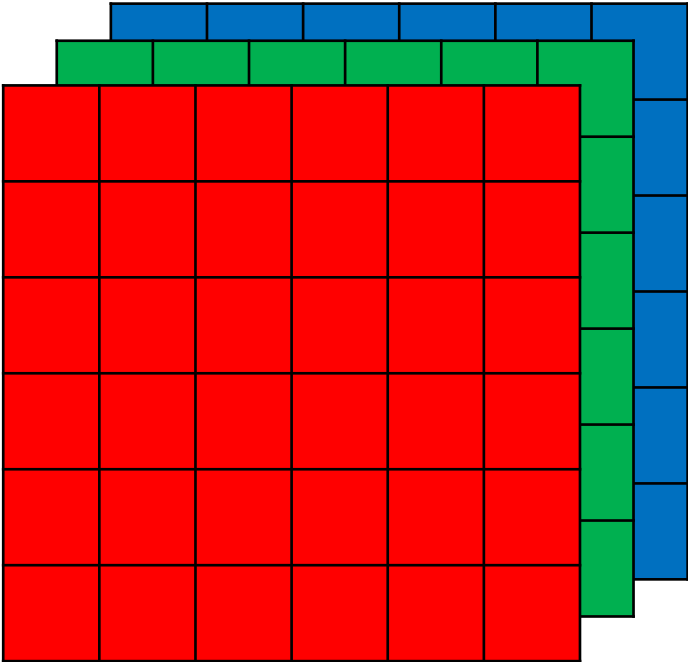
Convolutions over volumes

Convolutions on RGB images

Convolutions on RGB image

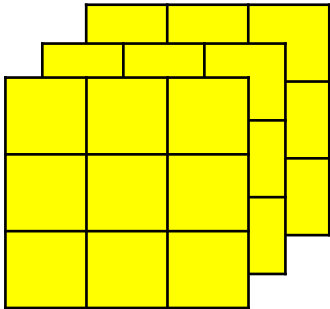


Multiple filters



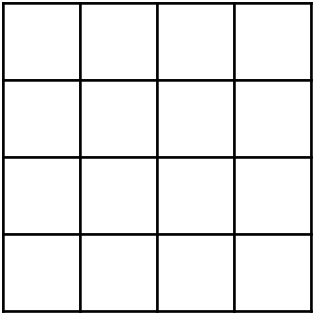
6 x 6 x 3

*



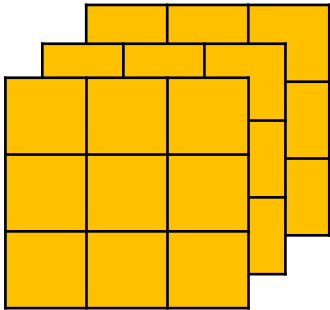
3 x 3 x 3

=



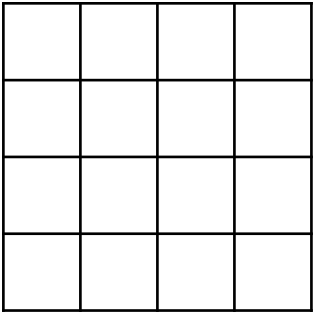
4 x 4

*



3 x 3 x 3

=



4 x 4



deeplearning.ai

Convolutional Neural Networks

A simple convolution
network example

Example ConvNet

Types of layer in a convolutional network:

- Convolution
- Pooling
- Fully connected



deeplearning.ai

Convolutional Neural Networks

Pooling layers

Pooling layer: Max pooling

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

Pooling layer: Max pooling

1	3	2	1	3
2	9	1	1	5
1	8	2	3	2
8	3	5	1	0
5	6	1	2	9

Pooling layer: Average pooling

1	3	2	1
2	9	1	1
1	4	2	3
5	6	1	2



Summary of pooling

Hyperparameters:

f : filter size

s : stride

Max or average pooling



deeplearning.ai

Convolutional Neural Networks

Convolutional neural network example

Neural network example

Neural network example

[illegible]



deeplearning.ai

Convolutional Neural Networks

Why convolutions?

Why convolutions

Why convolutions

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

 $*$

1	0	-1
1	0	-1
1	0	-1

 $=$

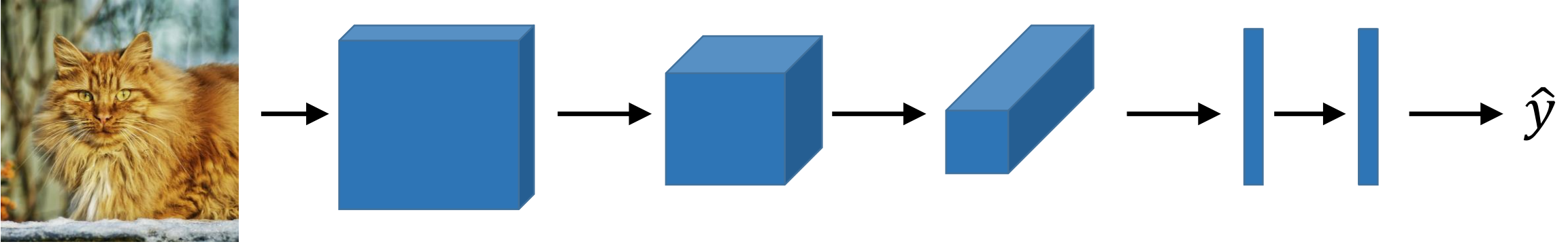
0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

Parameter sharing: A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

Sparsity of connections: In each layer, each output value depends only on a small number of inputs.

Putting it together

Training set $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$.



$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Use gradient descent to optimize parameters to reduce J

